

MODULE 01

SYLLABUS: Introduction to Operating Systems, Operating System Services, Overview of Operating Systems and Kernels, Linux Versus Classic Unix Kernels. Process concepts: Process Creation, Process States, Data Structures, Process API, Sharing processor among processes - user and kernel modes, context switching, System boot sequence. Case study: Linux kernel process management. Threads and Concurrency: Concept of a thread, Multithreading benefits, Multithreading models, Case study: The Linux Implementation of Threads. Process scheduling: Concepts and basic algorithms, The Multilevel Feedback Queue: Basic Rules, Case study: The Linux Completely Fair Scheduler (CFS). The Linux Scheduling Implementation, Preemption and Context Switching.

TO WATCH VIDEO CLASSES
OF THIS COURSE: [TAP HERE OR](#)
[SCAN QR CODE](#)



INTRODUCTION

Why an OS is Needed?

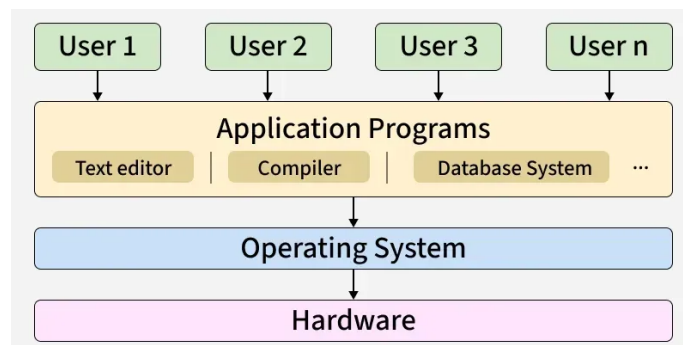
A program stored on disk must be loaded into RAM before the CPU can execute it. The CPU follows a **Fetch** → **Decode** → **Execute** cycle on instructions in memory. When multiple programs need to run simultaneously, critical questions arise — who gets the CPU? Who occupies memory? What if two programs access the same device? These problems make an OS necessary.

Without an OS: Only one program runs at a time; CPU stays idle during I/O; programs conflict over memory and devices; no file organization or security exists.

With an OS: Multitasking is enabled; memory is protected and managed; I/O is safely controlled; file systems provide organized persistent storage; security is enforced.

Definition

An **Operating System** is system software that acts as an interface between the user and hardware, managing all hardware resources efficiently, safely, and conveniently.



The OS plays three fundamental roles:

Role	What it means
Virtual Machine	Gives each program the illusion it owns the entire hardware
Interface (System Call Mechanism)	Provides a safe, controlled entry point between programs and hardware
Resource Manager	Allocates CPU, memory, and I/O fairly among competing processes

Virtualization — The Core OS Technique

Virtualization is the process by which the OS transforms a single physical resource into multiple virtual instances, giving each process the illusion of exclusive ownership.

Physical Resource	Virtualized As
Single CPU	Multiple virtual CPUs (via process scheduling)
Physical Memory	Private virtual address space per process
Disk	Files and directories
I/O Devices	Abstract, uniform device interfaces

1. CPU Virtualization: The OS rapidly switches the CPU among processes (time-sharing), creating the illusion of simultaneous execution. When multiple processes are ready, the **scheduler** decides which runs and for how long.

2. Memory Virtualization: Each process gets its own **virtual address space**, mapped to physical memory by the OS. One process cannot read or corrupt another's memory. Two processes may report the same virtual address yet operate on completely independent physical locations.

3. Persistence: RAM is volatile — data is lost on power-off. The **file system** stores data reliably on disk. Unlike CPU/memory, the OS does not create a private disk per process — the file system is shared across processes. File I/O uses three system calls: `open()` → `write()` → `close()`. For reliability, file systems use **journaling** or **copy-on-write**. For performance, writes are **batched**.

Concurrency

Concurrency refers to problems arising when multiple threads or processes execute simultaneously and share resources.

Root cause — Non-atomicity: The operation `counter++` appears simple but involves three machine instructions: *load* → *increment* → *store*. If two threads interleave between these steps without coordination, both may read the same old value, causing a **race condition** — the final result is incorrect and non-deterministic.

Example: Two threads each increment a shared counter 100,000 times. Expected: 200,000 | Actual: 143,012 or 137,298 (varies each run). Fix: Use synchronization primitives — locks, semaphores (covered in Module 2).

Program vs. Process

Aspect	Program	Process
Definition	Set of instructions stored on disk	Program in execution
Nature	Passive	Active
State	No execution state	Has state: ready, running, waiting
Resource usage	None	Uses CPU, memory, I/O
Lifetime	Permanent until deleted	Exists only during execution

Design Goals of an OS

User-Oriented Goals:

- **Convenience** — Hide hardware complexity; provide simple interfaces (GUI/CLI)
- **Performance** — Fast response time, high throughput, efficient CPU/memory use
- **Reliability** — Detect errors, maintain stability, recover from failures gracefully
- **Security & Protection** — Authentication, access control, process isolation

System-Oriented Goals:

- **Resource Utilization** — Maximize CPU, memory, I/O efficiency
- **Fairness** — All processes get fair share; prevent starvation
- **Scalability** — Perform efficiently as users, processors, or memory grow
- **Portability & Maintainability** — Adapt to different hardware; support modular evolution

Evolution / Types of Operating Systems

1. Batch OS: Jobs collected in batches and executed sequentially without user interaction. Simple but CPU idles during I/O; no interactivity. (*Example: IBM OS/360*)

2. Multiprogramming OS: Multiple programs loaded in memory; when one waits for I/O, the CPU switches to another. Improves CPU utilization but no user interaction. (*Example: Early UNIX*)

3. Time-Sharing OS: CPU time divided into small quanta; processes switched rapidly giving each user an interactive experience. Preemptive scheduling; fast response time. Overhead from frequent context switching. (*Example: UNIX, Linux, Windows*)

4. Real-Time OS (RTOS): Guarantees task completion within a strict time deadline. Correctness = Logical result + Timing constraint.

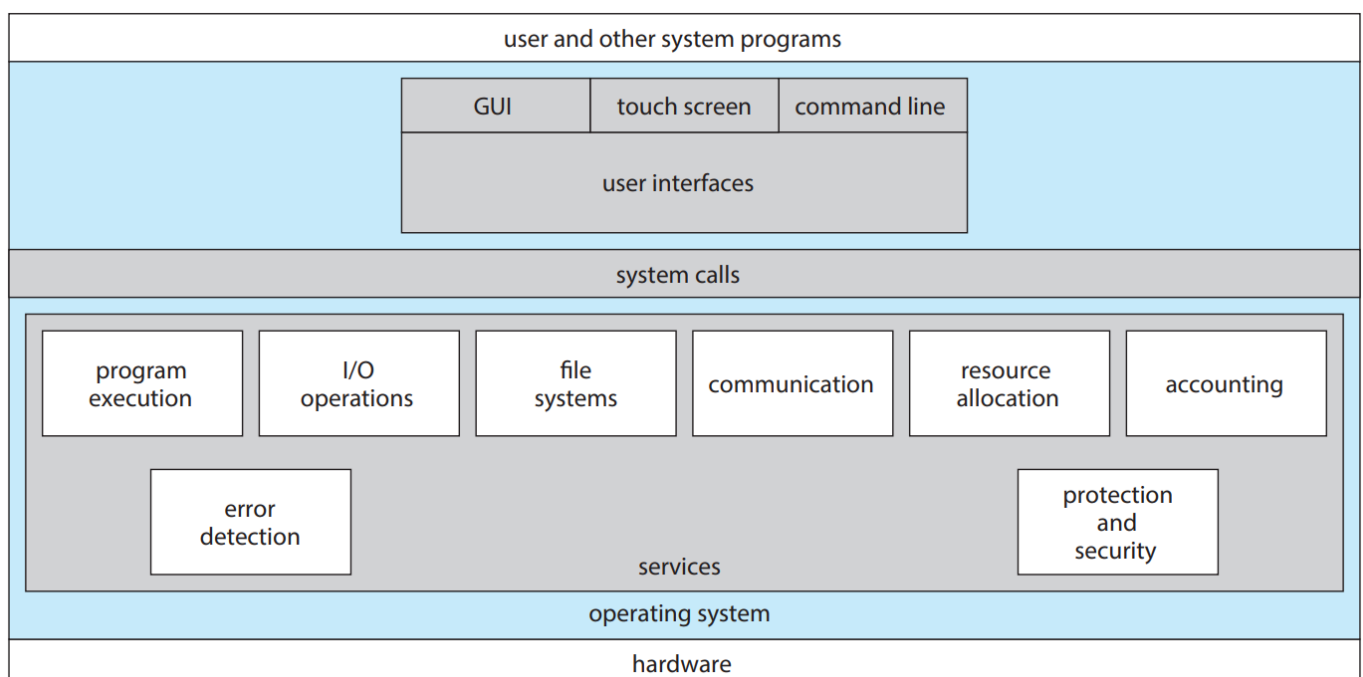
Aspect	Hard Real-Time	Soft Real-Time
Deadline	Must be met strictly	Preferable, not mandatory
Missing deadline	System failure	Performance degradation
Examples	Avionics, medical life-support	Video streaming, telecom

5. Distributed OS: Manages multiple independent computers over a network, presenting them as one unified system. Offers resource sharing, fault tolerance, and scalability. (*Example: Amoeba, Plan 9*)

6. Modern OS: Combines multitasking, multithreading, virtual memory, GUI, networking, security, and power management. (*Example: Linux, Windows 10/11, Android*)

Operating System Services

OS services are facilities provided to users and programs to make the system convenient, efficient, and safe.



User-Oriented Services

Service	Description
User Interface	CLI (terminal), GUI (windows/icons), Touch interface
Program Execution	Load program into memory, allocate resources, execute, handle termination
I/O Operations	Uniform, safe access to devices; handles buffering, caching, interrupts
File System Manipulation	Create/delete/read/write files and directories; manage permissions
Communication (IPC)	Data exchange between processes via shared memory or message passing
Error Detection	Detect CPU, memory, I/O, and program errors; take corrective action

System-Oriented Services

Service	Description
Resource Allocation	Assign CPU time, memory, devices among competing processes fairly
Accounting (Logging)	Track resource usage per process/user; used for billing and analysis
Protection & Security	User authentication, access control, process isolation, prevent misuse

Overview of OS and Kernels

Major OS Components

Component	Purpose
Kernel	Core — manages hardware directly, always in memory
Device Drivers	Control specific hardware devices
System Libraries	Provide APIs; applications call these
System Utilities	Maintenance and configuration tools
User Interface	CLI / GUI for user interaction

The Kernel

The **kernel** is the innermost core of the OS, running in privileged mode, always resident in memory, and directly controlling all hardware. Also called: **supervisor**, **core**, or **internals** of the OS.

Kernel functions:

Function	Explanation
Process Management	Creation, scheduling, termination of processes
Memory Management	Allocation, protection, virtual memory
I/O Management	Device drivers, interrupt handling
File System Control	Access control, buffering
System Call Handling	Controlled entry point to OS services
Protection & Security	Enforces access rules between processes

Kernel Space vs. User Space

Modern OSs separate execution into two protection domains:

Aspect	User Space	Kernel Space
Who runs here	User applications	OS kernel and device drivers
CPU Mode	User mode	Kernel mode
Hardware Access	Not permitted	Full, unrestricted
Memory Access	Own virtual address space only	Full physical memory
Effect of error	Process crashes (safe)	Can crash entire system

This separation provides **security**, **fault isolation**, and **system stability**.

Mode Bit

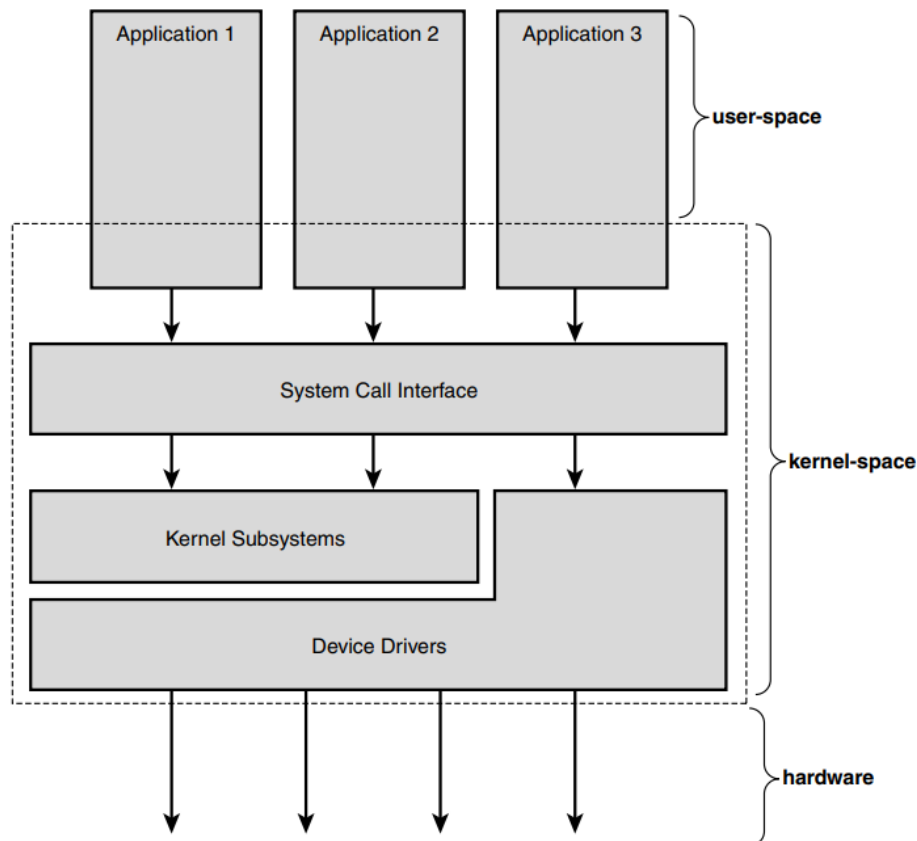
The **mode bit** is a hardware register bit that indicates the current CPU privilege level.

- **Mode bit = 0** → **Kernel mode**: Full hardware access; privileged instructions allowed
- **Mode bit = 1** → **User mode**: Restricted access; cannot directly touch hardware

The mode bit prevents user programs from executing privileged operations, protecting the OS and other processes.

System Calls and Execution Flow

A **system call** is the controlled mechanism by which a user program requests a service from the kernel.



Execution flow:

1. User program executes in user mode
2. Invokes system call instruction (**trap**)
3. Hardware switches CPU to kernel mode (mode bit → 0)
4. Kernel executes the requested service
5. **Return-from-trap** restores user mode (mode bit → 1); control returns to program

System Call Categories:

Category	Purpose	Examples
Process Control	Create/terminate processes	fork(), exec(), exit(), wait()
File Management	Create/read/write/delete files	open(), read(), write(), close()
Device Management	Control I/O devices	ioctl(), read(), write()
Information Maintenance	Get/set system info	getpid(), time(), uname()
Communication (IPC)	Message passing, shared memory	pipe(), shmget(), send(), recv()
Protection	Access rights, permissions	chmod(), chown(), setuid()

Kernel Execution Contexts

Context	Trigger	Can Block?	Has Process?
User Context	Normal program execution	Yes	Yes
Kernel Process Context	System call — kernel runs on behalf of a process	Yes	Yes
Interrupt Context	Hardware interrupt — not tied to any process	No	No

Linux vs. Classic Unix Kernels

Unix — Origin and Design

Unix was created in **1969** at Bell Labs by **Dennis Ritchie and Ken Thompson**, evolving from the failed Multics project. Rewritten in **C in 1973**, making it portable across hardware architectures.

Unix design strengths:

- Simple design — only hundreds of system calls
- **"Everything is a file"** — uniform interface: open(), read(), write(), lseek(), close()
- Portable (written in C)
- Fast process creation via fork()
- Clean separation of **policy** (what) and **mechanism** (how)

Unix Philosophy:

1. Do one thing and do it well
2. Programs should work together (via pipes)
3. Use simple, text-based interfaces
4. Everything is treated as a file

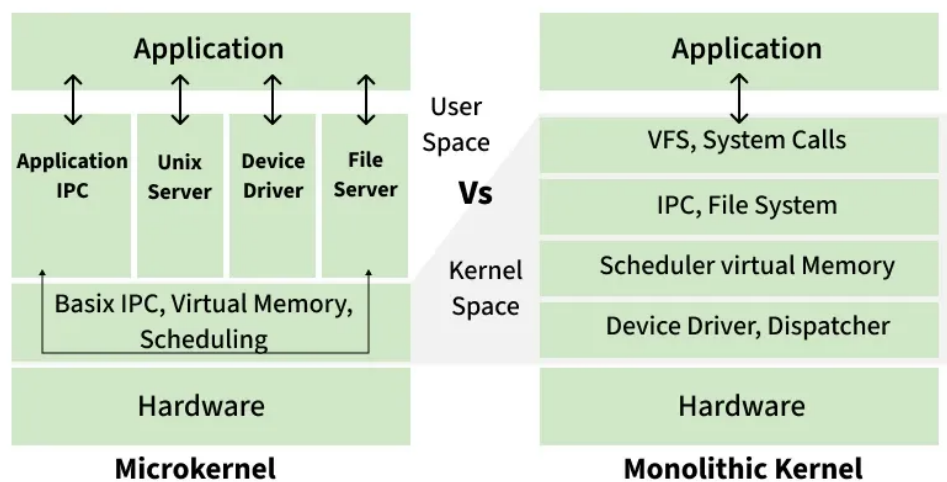
Linux — Introduction

Linus Torvalds created Linux in **1991** for Intel 80386, while a student at University of Helsinki. Written from scratch — implements the **Unix API (POSIX)** but shares no Unix source code.

- Licensed under **GNU GPL v2** — free and open-source
- Anyone may use, modify, and redistribute, provided they preserve the same freedoms
- Linux is **Unix-like, not Unix**
- Runs from embedded watches to supercomputers; powers Android and dominates cloud/servers

Monolithic Kernel vs. Microkernel

Feature	Monolithic Kernel	Microkernel
Structure	All OS services in one address space	Kernel minimal; services run as user-space servers
Communication	Direct function calls	Message passing via IPC
Performance	High — no IPC overhead	Lower — context switch + IPC latency
Fault Tolerance	Bug can crash entire kernel	Server crash doesn't bring down others
Complexity	Simpler internally	More complex due to IPC
Examples	Linux, traditional Unix	Mach (macOS base), Windows NT (theoretically)



Note: Windows NT and macOS moved microkernel servers back into kernel space for performance, largely defeating the microkernel purpose in practice.

Linux — Monolithic with Microkernel Benefits

Linux is **monolithic** (executes entirely in kernel space, uses direct function calls — no IPC overhead). However, it uniquely adopts microkernel advantages:

Linux Feature	Explanation
Dynamic Loadable Kernel Modules (LKM)	Kernel features loaded/unloaded at runtime — no reboot needed
Kernel Preemption	Kernel can preempt itself; higher-priority task interrupts even kernel-mode execution
SMP Support	Efficiently utilizes multiple CPUs simultaneously
Kernel Threads	Threads running entirely in kernel space
No thread/process distinction	Both are tasks internally; threads just share more resources
Object-oriented device model	Device classes, hot-plug events, sysfs filesystem
GPL v2 licensed	Free, open, community-driven

Linux vs. Classic Unix — Key Differences

Aspect	Linux	Classic Unix
Kernel type	Monolithic + modular	Pure monolithic
Kernel modules	Dynamic loading/unloading supported	Mostly static
Kernel preemption	Yes — preemptive kernel	Mostly non-preemptive
Thread handling	Threads = processes (tasks)	Threads and processes treated separately
SMP support	Strong	Limited in traditional Unix
Extensibility	High (modular design)	Less extensible
License	GPL v2 — open source	Proprietary variants
MMU requirement	Can run without MMU (special builds)	Requires MMU

Linux Kernel Versioning

Format: **Major . Minor . Revision . Stable**

Example: 2.6.26.1

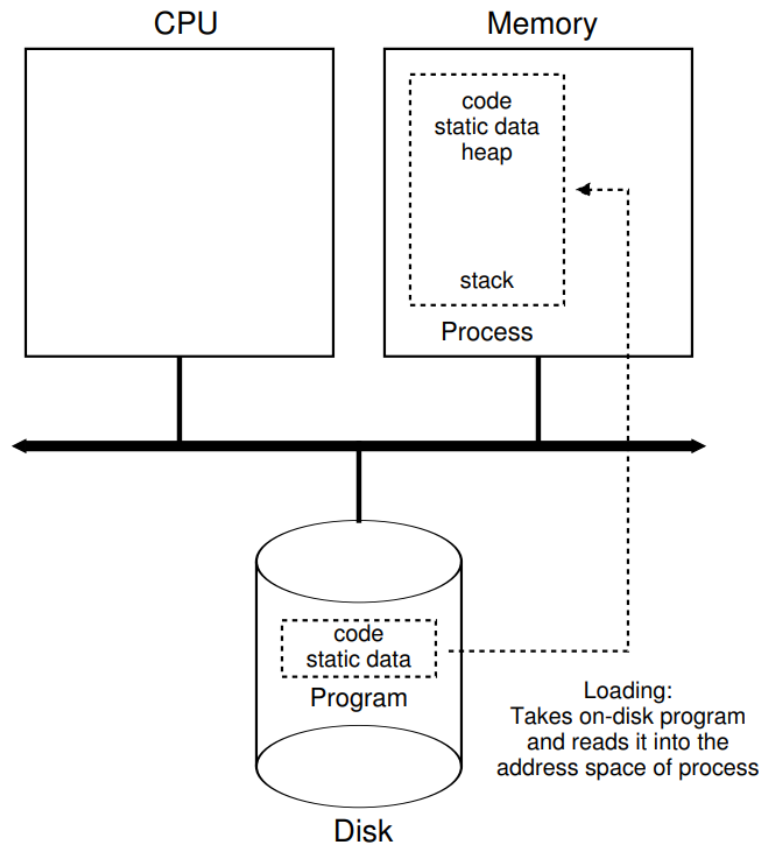
Field	Value	Meaning
Major	2	Major version
Minor	6	Even = Stable , Odd = Development
Revision	26	Bug-fix / feature revision
Stable	1	Stable patch number

Development kernels undergo rapid experimental changes and eventually stabilize into a new stable release. Modern Linux uses a time-based release model where the strict even/odd rule no longer applies.

PROCESS CONCEPTS

A **process** is a program in execution. A program stored on disk is passive; once loaded into memory and executing, it becomes an active process. A process is more than just code — it includes:

1. **Text section** — the executable program code
2. **Data section** — global and static variables
3. **Heap** — dynamically allocated memory (grows upward)
4. **Stack** — local variables, function parameters, return addresses (grows downward)
5. **Process state** — current activity (running, waiting, etc.)
6. **CPU registers** — program counter, stack pointer, general-purpose registers
7. **Open files and I/O information**



Stack holds temporary data per function call; heap holds dynamically allocated data; data section holds globals; text section holds the actual machine instructions.

Program vs. Process

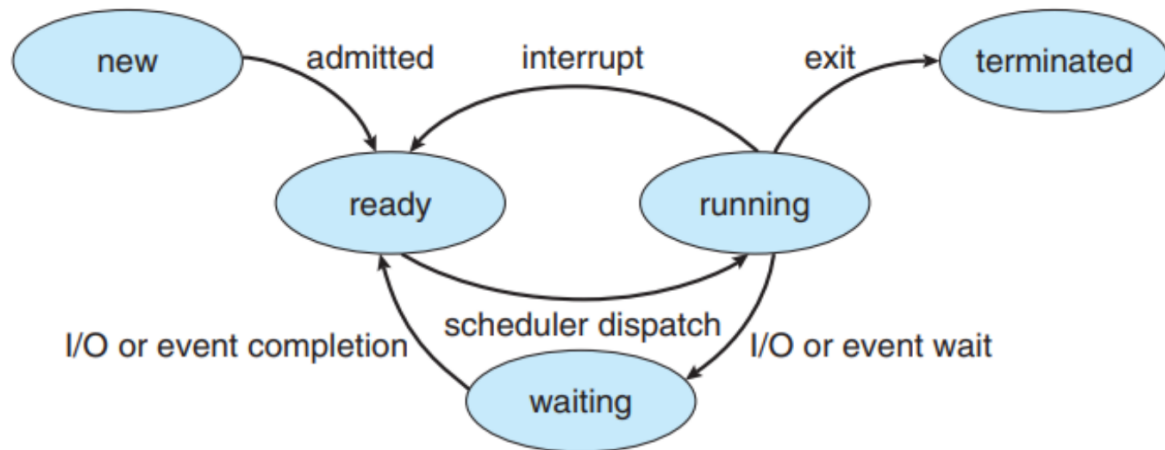
Aspect	Program	Process
Definition	Set of instructions stored on disk	Program in execution (active)
Nature	Passive	Active
State	No execution state	Has state: ready, running, waiting
Resource usage	None	Uses CPU, memory, I/O
Existence	Until deleted	Only during execution

Two or more processes can run the same program independently, each with its own state and resources.

Process States

At any instant, a process is in exactly one of the following states:

State	Description
New	Process is being created
Ready	Process is loaded in memory, waiting for CPU assignment
Running	Instructions are actively being executed on the CPU
Waiting (Blocked)	Process is waiting for an event — I/O completion, signal, etc.
Terminated	Process has finished execution; resources being freed



New becomes Ready when admitted by OS. Scheduler dispatches Ready → Running. An interrupt sends Running back to Ready. I/O request sends Running → Waiting; on I/O completion, Waiting → Ready. exit() sends Running → Terminated.

Model Question:

On a system with n processors, what is the minimum and maximum number of processes that can be in the ready, running and blocked states? (3 Marks)

Process State	Minimum Number	Maximum Number
Running	0	n
Ready	0	Infinite
Blocked	0	Infinite

Explanation

- In a system with n processors, at most n processes can run simultaneously.
- Any number of processes can remain in ready or blocked states depending on system load and I/O waiting.

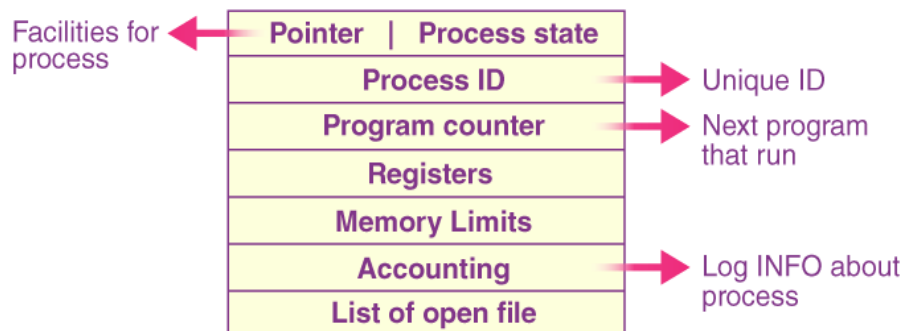
Process Data Structures

1. Process Control Block (PCB)

The **PCB** (also called **Task Control Block**) is the kernel data structure that stores all information about a process. Every process has one PCB, created when the process is created and deleted when it terminates.

PCB fields:

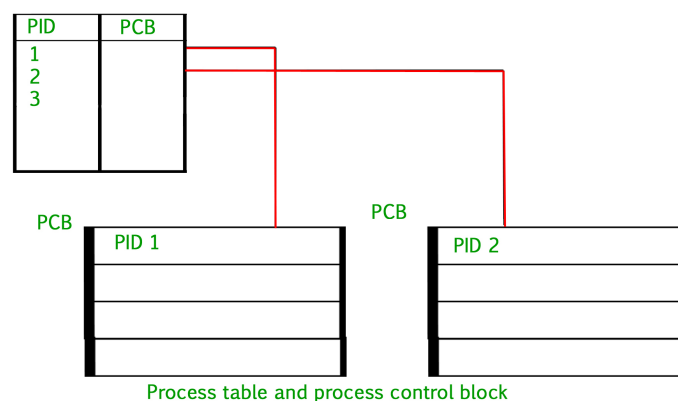
Field	Description
Process State	Current state: new, ready, running, waiting, terminated
Process ID (PID)	Unique integer identifier for the process
Program Counter	Address of the next instruction to execute
CPU Registers	Accumulators, index registers, stack pointers — saved on context switch
CPU Scheduling Info	Priority, pointers to scheduling queues
Memory Management Info	Base/limit registers, page tables or segment tables
I/O Status Info	List of open files, allocated I/O devices
Accounting Info	CPU time used, time limits, account numbers



Every field in the PCB is what the OS saves to "freeze" a process and restore it later exactly where it left off — this is what makes context switching possible.

2. Process Table

The **Process Table** is a kernel data structure (array or linked list of PCBs) that maintains information for all active processes. Its primary role is to **map a PID to its corresponding PCB**, allowing the OS to quickly locate any process.



Note: In Linux, the PCB equivalent is `struct task_struct`, defined in `<linux/sched.h>`. All processes are stored in a **circular doubly linked list** called the **task list**. Each element is a `task_struct`. Linux refers to processes as **tasks** internally. We look detailed in case study.

Process Creation

How a Process is Created (Loading)

Before a program runs, the OS must:

1. Load the program's code and static data from disk into memory (RAM)
2. Allocate a **stack** (for local variables, function params, return addresses)
3. Allocate a **heap** (for dynamic memory via malloc)
4. Initialize I/O (e.g., open stdin, stdout, stderr file descriptors)
5. Locate and jump to main() — the entry point

fork() and exec() — Unix Process Creation

Unix separates process creation into two steps, unlike other OSes that use a single "spawn":

fork() — creates a child process as a near-exact copy of the parent.

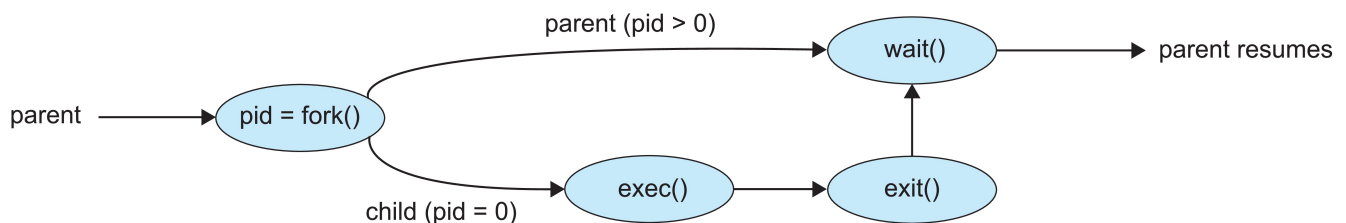
- Child gets a copy of the parent's address space (via **Copy-on-Write**)
- Both parent and child continue execution from the point after fork()
- fork() returns **twice**: returns child's PID (>0) to parent, returns **0** to child
- Child differs from parent only in: PID, PPID, and certain statistics

exec() — loads a new program into the child's address space and begins executing it.

- Replaces the current process image with a new program
- Does not create a new process — transforms the existing one

exit() — terminates the process and frees all its resources.

wait() — parent waits for child to finish; retrieves child's exit status.



After fork(), the parent typically calls wait() to block until the child finishes. The child calls exec() to load a new program, then exit(). When child exits, parent's wait() returns.

Copy-on-Write (COW)

Traditionally, fork() would copy all parent memory to the child — wasteful if exec() is called immediately after. Linux avoids this with **Copy-on-Write**:

- Parent and child initially **share** the same physical pages (marked read-only)
- A copy is made only when either process **writes** to a page
- If exec() is called immediately, no copying ever occurs
- This makes fork() very fast — only page tables and the new task_struct are duplicated

Model Question

On a Linux based system. a process holding an array of 1 million integers executes fork(). The child only reads the array to compute a sum, while the parent continues modifying the array. If the parent modifies 10% of the pages and the array spans 2000 pages, how many physical copies of the array exist immediately after fork()? How many pages get duplicated? (3 Marks)

Solution

After `fork()`, Linux uses **Copy-On-Write (COW)**.

- Parent and child initially share the same physical memory pages.
- Pages are copied only when modified.

Given

- Total pages = 2000
- Parent modifies = 10%

10 percent of 2000 = 200 pages

Answer

- Physical copies immediately after `fork()` = 1
- Number of duplicated pages = **200 pages**

Process Hierarchy (Process Family Tree)

All processes in Unix/Linux form a tree rooted at the **init process (PID = 1)**, which is started by the kernel at the end of boot. Every process has exactly one parent. A process can have zero or more children.

Processes with the same parent are called **siblings**.

- Each `task_struct` has a parent pointer and a children list
- If a parent exits before its children, the children are **reparented** to `init`
- `init` periodically calls `wait()` to clean up zombie children

Model Question

If the following code runs successfully, how many new processes get created? Draw a process tree depicting the hierarchy among the created processes.

```
int main(int argc, char *argv[])
{
    pid_t childpid = 0;
    int i;

    for (i = 1; i < 5; i++)
    {
        if ((childpid = fork()) ≤ 0)
            break;
    }

    return 0;
}
```

Solution

The loop runs for: $i = 1$ to 4

After `fork()`:

- Parent process gets `childpid > 0`
- Child process gets `childpid = 0`

Condition: (`childpid ≤ 0`)

Therefore:

- Child processes execute `break` and exit the loop immediately.
- Only the parent process continues looping and creates further children.

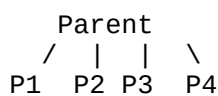
Thus:

- Each iteration creates exactly one new child process.
- Child processes do not create further children because they break out of the loop immediately.

Iteration-wise Process Creation

Iteration	New Process Created
i = 1	P1
i = 2	P2
i = 3	P3
i = 4	P4

Total New Processes Created: 4
(Excluding original parent process)

Process Tree

- Parent creates all child processes directly.
- Child processes do not create further children because of break.

Process Termination

A process terminates either voluntarily (calls `exit()`) or involuntarily (receives a signal it cannot handle). On `exit()`, the kernel:

1. Sets the `PF_EXITING` flag in `task_struct`
2. Releases memory (`exit_mm()`), open files (`exit_files()`), filesystem info (`exit_fs()`)
3. Sends a signal to the parent
4. Sets the process state to **EXIT_ZOMBIE**
5. Calls `schedule()` — the process never runs again

Zombie Process: After termination, a process enters the zombie state. It still occupies a `task_struct` (to hold exit status for the parent). Once the parent calls `wait()`, the zombie is cleaned up and the `task_struct` is freed via `release_task()`.

Orphan Process: If a parent exits before its children, the children become orphans. Linux reparents them to another thread in the thread group, or to `init`. `init` routinely calls `wait()` to reap them.

Process API

The OS provides the following process-related API (system calls):

API Category	Purpose	System Calls
Create	Create a new process	<code>fork()</code> , <code>exec()</code>
Destroy	Force-terminate a process	<code>exit()</code> , <code>kill()</code>
Wait	Wait for a process to stop	<code>wait()</code> , <code>waitpid()</code>
Miscellaneous Control	Suspend, resume, get info	<code>suspend()</code> , <code>getpid()</code> , <code>getppid()</code>
Status	Get process state/resource usage	<code>getpid()</code> , <code>ps</code> , <code>/proc</code>

Sharing the Processor Among Processes

To virtualize the CPU, the OS time-shares it among processes. The basic approach: run one process for a short time, then switch to another. This creates the illusion of parallelism. Two key challenges arise:

- **Performance** — How to implement switching without excessive overhead
- **Control** — How to prevent a process from running forever or accessing forbidden resources

Limited Direct Execution

The OS runs programs directly on the CPU for performance. Before running, the OS:

1. Creates a process list entry
2. Allocates memory
3. Loads program into memory
4. Sets up stack with argc/argv
5. Clears registers and calls main()

Problem: Without limits, a process could do anything — access all memory, run forever, perform unauthorized I/O.

Solution: Introduce two CPU modes.

User Mode and Kernel Mode

Aspect	User Mode	Kernel Mode
Who runs here	User applications	OS kernel
Hardware access	Restricted — no direct access	Full access
Privileged instructions	Not allowed	Allowed
Direct I/O	Not allowed	Allowed
Entry	Normal	Via system call (trap) or interrupt

Mode Bit: A hardware register bit in the CPU status register:

- **Mode bit = 1** → **User mode** (restricted)
- **Mode bit = 0** → **Kernel mode** (privileged)

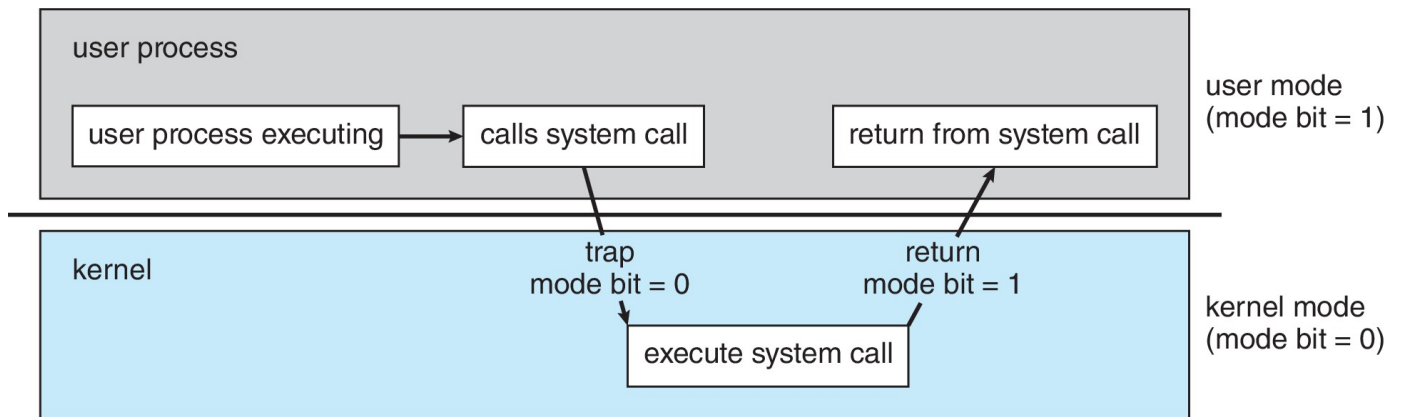
Example: Notepad opens a file → runs in user mode → calls open() (system call) → CPU sets mode bit to 0 (kernel mode) → OS accesses disk → returns result → CPU sets mode bit back to 1 (user mode).

System Call — Trap Mechanism

When a user program needs a privileged operation, it executes a **trap instruction**:

1. Jumps into the kernel and sets mode bit to 0 (kernel mode)
2. Kernel executes the requested service
3. **Return-from-trap** instruction restores mode bit to 1 (user mode) and returns control

Trap Table: At boot time (in kernel mode), the OS sets up a **trap table** — a hardware-level table that maps each system call number to its handler address. Hardware remembers this table until next reboot.



A user process cannot enter kernel mode directly. Only the trap instruction (system call) can do so in a controlled way, jumping to the pre-set trap handler.

Model Question

A hypothetical operating system provides a system call that allows a process to request for memory allocation. An experienced programmer gives the following advice: “If your program makes many small requests for memory, you can improve its performance by combining them into a single request and making only one system call.” Argue why the advice makes sense. (3 Marks)

Solution

The advice makes sense because every system call causes overhead.

When a process requests memory:

- The CPU switches from user mode to kernel mode.
- The operating system executes memory management operations.
- Then control returns to user mode.

If many small memory requests are made:

- Many system calls occur.
- This increases context switching and kernel overhead.

By combining multiple small requests into one large request:

- The number of system calls is reduced.
- Less overhead occurs.
- Program execution becomes faster and more efficient.

Hence, a single large memory allocation improves performance compared to many small allocations.

Switching Between Processes — Regaining CPU Control

Problem: How does the OS regain the CPU from a running process to switch to another?

1. Cooperative Approach: OS trusts processes to give up the CPU voluntarily — by making system calls or on errors (division by zero, illegal memory access). Used in early systems. **Flaw:** An infinite loop with no system calls will starve all others.

2. Non-Cooperative Approach (Timer Interrupt): A hardware timer is programmed to fire an interrupt every few milliseconds. When it fires:

- The current process is halted
- The OS's pre-configured **interrupt handler** runs
- The OS regains control and can switch to another process

Modern OSes use the timer interrupt approach, making the OS truly in control at all times.

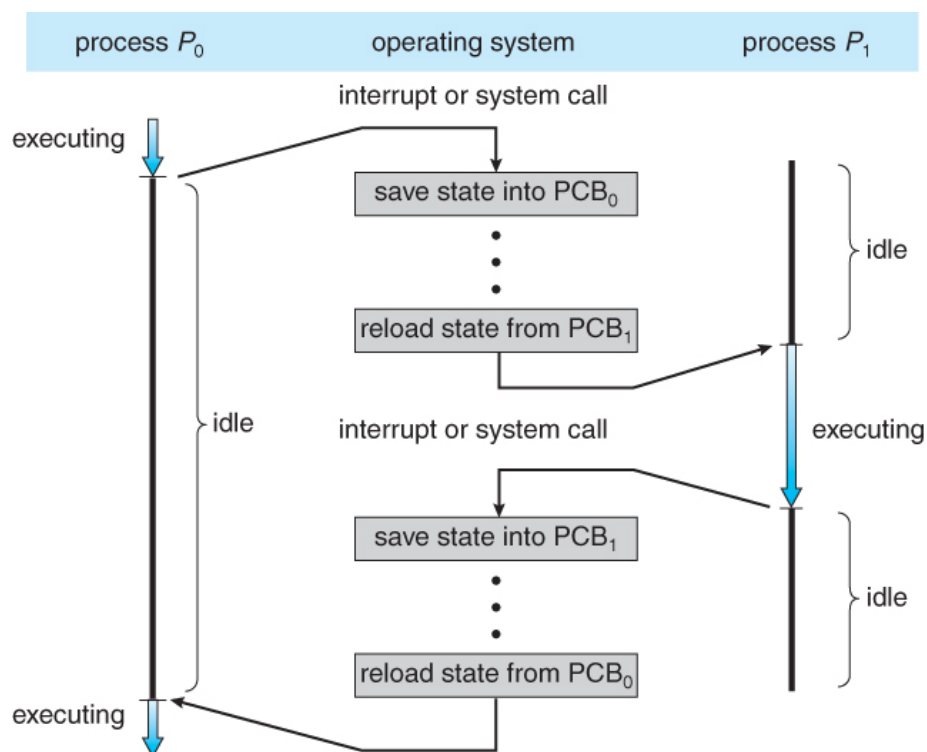
Context Switching

Context switching is the process by which the OS saves the state of the currently running process and loads the state of another process, enabling the CPU to switch between processes.

The **context** of a process includes everything needed to resume it: CPU registers (program counter, stack pointer, general-purpose registers), process state, and memory management information — all stored in the PCB.

Steps in a Context Switch:

1. OS saves current process state into its PCB (PCB₀)
2. Updates current process status to Ready or Waiting
3. Scheduler selects the next process to run
4. OS loads the saved context of the new process from its PCB (PCB₁)
5. CPU resumes execution of the new process from where it was interrupted



About Fig:

- P_0 executing → interrupt/system call → OS saves state into PCB₀ → OS reloads state from PCB₁ → P_1 executing
- Later: P_1 interrupt → OS saves into PCB₁ → OS reloads from PCB₀ → P_0 executing again Show P_0 and P_1 as idle during their respective wait periods.

Context switch overhead is real — saving and restoring registers takes time. Frequent context switches improve responsiveness but reduce throughput. This trade-off drives scheduling policy design.

System Boot Sequence

The **boot sequence** is the step-by-step process by which a computer powers on, initializes hardware, and loads the operating system into memory.

Steps:

1. **Power ON** — CPU is reset; instruction register is loaded with a fixed ROM address
2. **Bootstrap Loader (ROM/BIOS/UEFI)** — small program stored in ROM executes; it knows how to access disk
3. **Diagnostics (POST)** — Power-On Self Test; checks CPU, RAM, and I/O devices for faults
4. **Initialization** — CPU registers, device drivers, and RAM contents are initialized

5. **Load OS** — Bootstrap loader reads the OS kernel from disk (HDD/SSD) into RAM, one component at a time
6. **Kernel starts** — Kernel initializes subsystems (memory manager, process manager, file system, device drivers)
7. **Init process** — Kernel launches init (PID 1), which starts all system services and the login interface

ROM is non-volatile; it holds the bootstrap loader permanently. The bootstrap loader is small — its only job is to find and load the OS. The OS itself then takes over all hardware initialization.

Case Study: Linux Kernel Process Management

Process in Linux

- In Linux, a process is called a **task**.
- All processes are stored in a circular doubly linked list called the **task list**.
- Each process is represented using a structure called **task_struct**.

Important fields of task_struct

Field	Purpose
state	Current state of process
pid	Process ID
parent	Pointer to parent process
children	List of child processes
mm	Address space
files	Open files
prio	Scheduling priority

Process States in Linux

State	Meaning
TASK_RUNNING	Running or waiting in ready queue
TASK_INTERRUPTIBLE	Sleeping; wakes on signal/condition
TASK_UNINTERRUPTIBLE	Sleeping; ignores signals
__TASK_TRACED	Process traced by debugger
__TASK_STOPPED	Process stopped

Important Difference

- **Interruptible** → wakes using signals
- **Uninterruptible** → ignores signals

Process Creation in Linux

Linux implements **fork()** using **clone()** system call.

Main functions:

- **do_fork()**
- **copy_process()**

Steps in Process Creation

1. Create new task_struct
2. Check resource limits
3. Assign new PID
4. Set child state
5. Copy/share resources
6. Wake up child process

Important Point: Linux often runs the **child first** after fork() to reduce copy-on-write overhead.

clone() Flags

Flag	Meaning
CLONE_VM	Share memory space
CLONE_FILES	Share open files
CLONE_FS	Share filesystem info
CLONE_SIGHAND	Share signal handlers
CLONE_THREAD	Same thread group
CLONE_VFORK	Parent waits for child

Important Note

- Normal process → shares almost nothing
- Thread → shares most resources

Threads in Linux

- Linux does not distinguish between threads and processes.
- Both use task_struct.
- Threads are processes that share resources.

Shared Resources

- Address space
- Open files
- Signal handlers

Advantage

- Simple and efficient design

Kernel Threads

- Kernel threads run completely inside kernel space.
- They do not have user address space.
- Used for background kernel tasks.

Examples:

- flush
- ksoftirqd

Process Termination in Linux

Handled using:

- exit()
- do_exit()

Steps

1. Release memory
2. Release files
3. Notify parent
4. Enter EXIT_ZOMBIE state
5. Scheduler removes process

Zombie Process

- Process finished execution but task_struct still exists.
- Parent must call wait() to remove it.

Reparenting

If parent dies before child:

- Linux assigns new parent process
- Usually init process

Conclusion

Linux process management is designed to be simple, efficient, and flexible. Every process and thread is represented using the same data structure called task_struct, which simplifies kernel design. Process creation is performed using fork() and clone(), allowing resource sharing between tasks. Linux efficiently manages process states, scheduling, threads, and termination mechanisms such as zombie handling and reparenting. This unified approach provides high performance and scalability in modern operating systems.

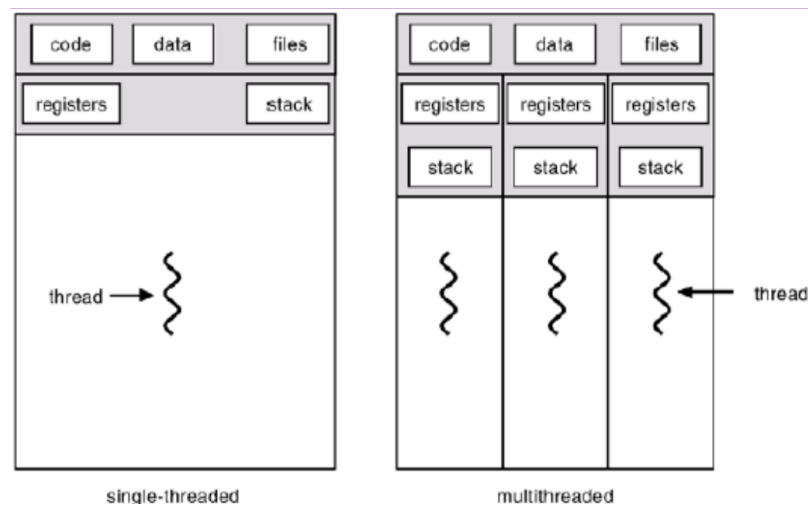
THREADS AND CONCURRENCY

A **thread** is the smallest unit of execution within a process. It is also called a **lightweight process**. A single process can contain multiple threads, all sharing the same address space, code, data, and open files — but each thread has its own program counter, register set, and stack.

The idea behind threads is to achieve **parallelism within a process** by dividing its work into multiple concurrent execution paths. For example, a browser can have one thread per tab; MS Word uses one thread to format text and another to handle keyboard input simultaneously.

What a Thread Has (Per-Thread Items) vs. What it Shares (Per-Process Items)

Per Process (Shared by all threads)	Per Thread (Each thread has its own)
Address space (code + data + heap)	Program Counter (PC)
Global variables	Register set
Open files	Stack
Child processes	Thread state
Pending signals and signal handlers	
Accounting information	



In a single-threaded process, there is only one execution path. In a multi-threaded process, code and data are shared but each thread has its own stack and registers — enabling concurrent execution paths within the same process.

Concurrency vs. Parallelism

These two terms are frequently confused in exams:

Aspect	Concurrency	Parallelism
Definition	Multiple tasks start, run, and complete in overlapping time periods — not necessarily at the same instant	Multiple tasks or subtasks run at the same instant on multiple CPU cores
Hardware needed	Single or multi-core	Requires multi-core processor
Execution	Interleaved (time-shared)	Truly simultaneous
Example	OS switching between threads on one CPU	Two threads running on two different cores simultaneously

Concurrency is about dealing with many things at once; Parallelism is about doing many things at once.

Process vs. Thread

Basis	Process	Thread
Definition	Program in execution (active)	Segment / execution unit of a process
Memory space	Separate address space	Shared address space within process
Communication	IPC required — more overhead	Direct via shared memory — faster
Context switch	Higher overhead (full address space switch)	Lower overhead (only registers/stack)
Creation/termination time	More time	Less time
Resource consumption	More	Less
Blocking	One blocked process doesn't affect others	One blocked user-level thread may block all peers
Independence	Fully independent	Dependent — share process resources
Data/code sharing	Independent data and code	Shares code, data, and files with peers
OS treatment	Each treated separately	All user-level threads treated as one task
Switching	Requires OS interface	May not require OS call
Parent-child	Parent changes don't affect children	Changes in main thread may affect all peer threads

Multithreading Benefits

Multithreading is the ability of a process to execute multiple threads simultaneously within the same program.

A thread is the smallest unit of execution inside a process. Multiple threads share the same memory space, files, and resources of the parent process, but each thread has its own program counter, registers, and stack.

Multithreading offers four major benefits, which are a standard exam question:

1. Responsiveness — If one thread is blocked (e.g., waiting for I/O), other threads in the same process continue running. An interactive application remains responsive even during a long operation. Example: a browser loading a page (one thread) still lets you type in the address bar (another thread).

2. Resource Sharing — Threads automatically share the memory and resources of their process. This allows multiple threads to work within the same address space without any explicit IPC mechanism — far more efficient than inter-process communication.

3. Economy — Creating a process involves allocating a new address space, duplicating parent memory (COW), and setting up a full PCB. Creating a thread is much cheaper since it reuses the existing process resources. Context switching between threads is also faster than between processes.

4. Scalability / Utilization of Multiprocessors — On a multi-core system, threads from the same process can run in true parallel on different cores. A single-threaded process can use only one core at a time, no matter how many cores the system has.

Disadvantages of Multithreading:

- Shared resources require proper synchronization — incorrect access leads to race conditions
- Multithreaded programs are harder to write, debug, and maintain
- A bug in one thread (e.g., corrupting shared data) can crash the entire process

Multithreading Models

Threads exist at two levels: **user-level threads (ULT)** — managed by a user-space library, invisible to the kernel — and **kernel-level threads (KLT)** — managed directly by the OS kernel. Multithreading models describe how user threads are **mapped to** kernel threads.

1. User-Level Threads (ULT)

Implemented entirely in a user-space library (e.g., POSIX pthreads). The kernel is unaware of their existence and treats the entire process as a single-threaded entity.

Advantages:

- No kernel involvement — thread creation, switching, and synchronization are all done in user space without system calls
- Fast context switching between threads
- Does not require OS modification

Disadvantage:

- If one ULT makes a blocking system call, the entire process (and all its threads) is blocked by the kernel — because the kernel sees only one thread

2. Kernel-Level Threads (KLT)

The kernel is fully aware of and manages all threads. The kernel maintains a **thread table** alongside the process table. Thread creation and management require system calls.

Advantages:

- Kernel can schedule threads on different processors simultaneously
- Blocking one thread does not block others — kernel schedules another
- Scheduler can give more CPU time to processes with more threads

Disadvantage:

- Thread creation, switching, and synchronization involve system calls — higher overhead than ULT

Model Question

An operating system supports both user-level threads and kernel-level threads. A developer proposes the following guidelines for deciding which type of thread to use:

G1: If a task is CPU-bound, it should be implemented as a kernel-level thread when the system has multiple processors; otherwise, it should be implemented as a user-level thread.

G2: If a task is I/O-bound, it should be implemented as a user-level thread when the process has no kernel-level threads; otherwise, it should be implemented as a kernel-level thread.

Do you agree or disagree with these guidelines? Briefly justify your answer.

(6 Marks)

Solution

G1: CPU-bound Tasks

Statement:

If a task is CPU-bound:

- Use kernel-level threads on multiprocessor systems
- Otherwise use user-level threads

Agree - This guideline is generally correct.

Justification

- Kernel-level threads can run truly in parallel on multiple processors.
- This improves CPU utilization and performance for CPU-intensive tasks.
- On a single processor, user-level threads are more efficient because they have lower overhead and faster context switching.

G2: I/O-bound Tasks

Statement

If a task is I/O-bound:

- Use user-level threads when no kernel-level threads exist
- Otherwise use kernel-level threads

Partially Agree - This guideline is only partially correct.

Justification

- In pure user-level threading, if one thread performs a blocking I/O operation, the entire process may block because the kernel is unaware of individual user threads.
- Kernel-level threads avoid this problem since the kernel can schedule another thread while one thread waits for I/O.
- Therefore, kernel-level threads are generally better for I/O-bound tasks.

However:

- User-level threads are still lightweight and efficient for applications with nonblocking or asynchronous I/O.

Conclusion

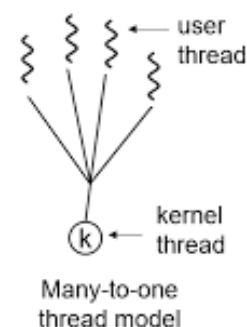
- G1 is mostly correct because kernel-level threads provide parallelism for CPU-bound tasks on multiprocessor systems.
- G2 is partially correct, but kernel-level threads are usually preferred for I/O-bound tasks because blocking in one thread does not stop the entire process.

Multithreading Models

1. Many-to-One Model

Multiple user threads are mapped to **one** kernel thread.

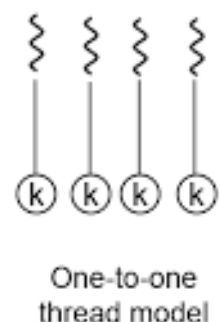
- Thread management done in user space — fast
- If one thread makes a blocking call, the entire process blocks
- Cannot utilize multiple processors (only one kernel thread)
- Example: Early Green Threads (Java)



2. One-to-One Model

Each user thread maps to **one** unique kernel thread.

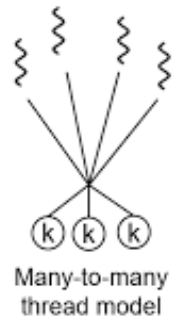
- Blocking one thread does not block others
- True parallelism on multi-core systems
- Overhead is high — creating a user thread requires creating a kernel thread
- Example: Linux (via clone()), Windows



3. Many-to-Many Model

Multiple user threads are **multiplexed** onto a smaller or equal number of kernel threads.

- Best of both worlds: no blocking problem + true parallelism possible
- If one thread blocks, the library can schedule another user thread on another kernel thread
- Most flexible and efficient model
- Example: Solaris, older versions of IRIX



Comparison

Feature	Many-to-One	One-to-One	Many-to-Many
Parallelism	No	Yes	Yes
Blocking impact	Entire process blocks	Only that thread blocks	Other threads can continue
Overhead	Low	High	Medium
Kernel awareness	None	Full	Partial
Example	Green Threads	Linux, Windows	Solaris

Case Study: Linux Implementation of Threads

Linux Approach to Threads

Linux does not distinguish between threads and processes at the kernel level. Both are represented using the same data structure called `task_struct`.

A thread in Linux is simply a process that shares resources with another process.

Shared Resources

- Address space
- Open files
- Signal handlers
- Filesystem information

Important Point: Linux treats threads as lightweight processes.

Creating Threads in Linux

Threads are created using the `clone()` system call with specific sharing flags.

Example

```
clone(CLONE_VM | CLONE_FS | CLONE_FILES | CLONE_SIGHAND, 0);
```

This creates a thread that shares resources with the parent process.

Important clone() Flags

Flag	Purpose
CLONE_VM	Share address space
CLONE_FS	Share filesystem information
CLONE_FILES	Share open file descriptors
CLONE_SIGHAND	Share signal handlers
CLONE_THREAD	Place threads in same thread group

Shared vs Private Resources

Shared Resources	Private Resources
Virtual address space	Stack
Open files	Registers
Signal handlers	Program counter
Filesystem information	Thread state
Heap and global data	task_struct

Thread Groups and TGID

When CLONE_THREAD is used:

- All threads belong to the same thread group.
- All threads share the same TGID (Thread Group ID).
- Each thread still has its own unique PID.

Important System Calls

- getpid() → returns TGID
- gettid() → returns individual thread ID

Kernel Threads

Kernel threads are special threads that run completely inside kernel space.

Features

- No user address space (mm = NULL)
- Fully schedulable
- Used for background kernel operations

Examples

- kworker, ksoftirqd, flush

Difference Between User Process and Kernel Thread

Feature	User Process	Kernel Thread
Address Space	Has user address space	No user address space (mm = NULL)
Execution Mode	Runs in user space and kernel space	Runs only in kernel space
Scheduling	Scheduled by CPU scheduler	Scheduled by CPU scheduler
Context Switch to User Space	Possible	Never occurs
Examples	User applications	kworker, ksoftirqd, flush

Linux Thread Model vs Classical Thread Model

Aspect	Classical OS (Windows/Solaris)	Linux
Thread Representation	Separate thread object	task_struct
Thread Creation	Dedicated thread API	clone()
Thread vs Process	Clearly separated	Both treated as tasks
Kernel Structures	Separate process/thread structures	Single structure
Efficiency	More overhead	Highly efficient

Advantages of Linux Thread Model

1. Simple kernel design
2. Efficient resource sharing
3. Less overhead
4. Faster thread creation
5. Better performance

Conclusion

Linux implements threads by treating them as processes that share resources. Both processes and threads use the same `task_struct` data structure, which simplifies kernel design and reduces overhead. Threads are created using the `clone()` system call with sharing flags, allowing efficient multithreading support. This unified approach makes Linux thread management simple, flexible, and highly efficient.

PROCESS SCHEDULING

Before studying scheduling algorithms, you must be clear on what a **CPU burst** is. Every process alternates between two phases: a **CPU burst** (period of active computation) and an **I/O burst** (period of waiting for input/output). The scheduler makes decisions during transitions between these bursts. Understanding this cycle is essential to appreciating why different algorithms exist.

CPU Scheduling

CPU scheduling is the process by which the operating system decides which process in the ready queue should be allocated the CPU for execution at any given moment.

The main objective is to keep the CPU busy at all times while ensuring fairness and good response times. Whenever the CPU becomes idle, the scheduler selects one process from the ready queue and dispatches it. The component that actually performs the switch is called the **Dispatcher**.

Scheduling Decision Points

A scheduling decision is made when a process:

1. Switches from **Running** → **Waiting** (e.g., I/O request) — non-preemptive
2. Switches from **Running** → **Ready** (e.g., timer interrupt) — preemptive
3. Switches from **Waiting** → **Ready** (e.g., I/O completion) — preemptive
4. **Terminates** — non-preemptive

Cases 1 and 4 force a scheduling decision; cases 2 and 3 offer a choice.

Types of Scheduling

Type	Description
Non-Preemptive	Once a process gets the CPU, it keeps it until it voluntarily releases (by blocking or terminating). No special timer hardware needed.
Preemptive	The OS can forcibly take the CPU away from a running process and give it to a higher-priority or more deserving one. Used in all modern OS including Linux.

Types of Schedulers

Long-Term Scheduler (Job Scheduler): Selects which processes should be brought from the job pool into the ready queue in memory. Invoked infrequently (seconds to minutes). Controls the degree of multiprogramming.

Short-Term Scheduler (CPU Scheduler): Selects which process in the ready queue gets the CPU next. Invoked very frequently (milliseconds). This is what we commonly mean by "the scheduler."

Medium-Term Scheduler: Manages swapping — temporarily removing processes from memory (swap out) and reintroducing them later (swap in). Handles blocked/waiting processes.

Dispatcher

The **Dispatcher** is the module that gives control of the CPU to the process selected by the short-term scheduler. It performs context switching, switching to user mode, and jumping to the correct location in the user program. The time it takes to do this is called **Dispatch Latency**, and it should be minimized.

Property	Scheduler	Dispatcher
Role	Decides which process runs next	Transfers CPU control to selected process
Types	Long, Medium, Short-term	Single module (no types)
Functions	Process selection, queue management	Context switch, mode change, process start
Time	Takes longer, invoked less often	Very fast (dispatch latency)

Scheduling Criteria and Key Terminologies

Performance Metrics

Term	Definition	Formula
Arrival Time (AT)	When process enters ready queue	—
Burst Time (BT)	Total CPU time needed by process	—
Completion Time (CT)	Time when process finishes	—
Turnaround Time (TAT)	Total time from arrival to completion	$TAT = CT - AT = WT + BT$
Waiting Time (WT)	Time spent waiting in ready queue	$WT = TAT - BT$
Response Time (RT)	Time from arrival to first CPU response	First execution time – AT

Optimization Goals

Maximize: CPU Utilization, Throughput (jobs/second)

Minimize: Turnaround Time, Waiting Time, Response Time

There is an inherent trade-off: minimizing response time (good for interactive systems) leads to more context switches, reducing throughput. A good scheduler balances both.

Classification of Processes

Before understanding algorithms, it is important to understand what kind of processes exist, as scheduling policy is heavily shaped by this.

I/O-Bound Processes spend most of their time waiting on I/O and have short CPU bursts. Examples: text editors, GUI applications. These need to be scheduled quickly to maintain good interactive response.

CPU-Bound (Processor-Bound) Processes spend most of their time executing and have long CPU bursts. Examples: mathematical computations, video encoding. These can tolerate waiting longer between CPU allocations.

Linux explicitly **favors I/O-bound processes** over CPU-bound ones to provide good interactive response, while still ensuring CPU-bound processes are not neglected.

Scheduling Algorithms

1. First Come First Served (FCFS)

Processes are scheduled in the order they arrive in the ready queue. It is a non-preemptive algorithm based on a FIFO queue.

FCFS is the simplest scheduling algorithm. The process that arrives first gets the CPU first and runs to completion before the next process starts. Since it is non-preemptive, no process is interrupted once it begins. While simple to implement, it performs poorly when a long CPU-bound process arrives before several short ones — the short processes are forced to wait, a phenomenon known as the **Convoy Effect**.

Convoy Effect: A situation where many short processes queue up behind one long process, causing poor CPU and I/O utilization. For example, if one CPU-bound process holds the CPU, all I/O-bound processes waiting behind it cannot proceed, causing the I/O devices to sit idle as well.

Example 1:

Process	Burst Time (BT)
P1	24
P2	3
P3	3

Arrival Order: P1 $\rightarrow$ P2 $\rightarrow$ P3

Gantt Chart:



Process	CT	TAT	WT
P1	24	24	0
P2	27	27	24
P3	30	30	27

Average Waiting Time = $(0 + 24 + 27) / 3 = 17$ ms

Average Turnaround Time = $(24 + 27 + 30) / 3 = 27$ ms

Example 2:

Process	Burst Time (BT)
P2	3
P3	3
P1	24

Arrival Order: P2 $\rightarrow$ P3 $\rightarrow$ P1

Gantt Chart:



Process	CT	TAT	WT
P1	30	30	6
P2	3	3	0
P3	6	6	3

Average Waiting Time = $(6 + 0 + 3) / 3 = 3$ ms

Average Turnaround Time = $(30 + 3 + 6) / 3 = 13$ ms

Advantages: Simple, easy to implement, no starvation (every process eventually executes).

Disadvantages: Convoy effect; poor average waiting time; non-preemptive so unsuitable for interactive systems.

2. Shortest Job First (SJF) — Non-Preemptive

The process with the shortest CPU burst time is scheduled next. It is non-preemptive — once a process starts, it runs to completion.

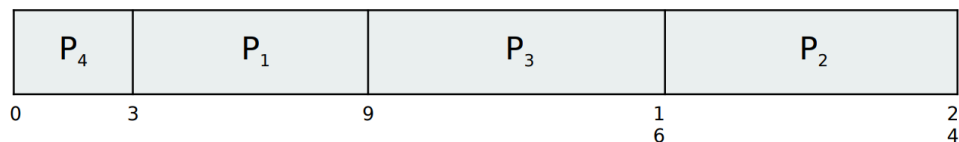
SJF is provably **optimal** in terms of minimizing average waiting time for a given set of processes. The idea is straightforward: by executing shorter jobs first, the number of processes waiting behind a long job is reduced. The major practical limitation is that the CPU burst time must be known in advance, which is possible in batch systems but not in interactive ones.

Example 1

Process	Burst Time (BT)
P1	6
P2	8
P3	7
P4	3

In SJF scheduling, the process with the smallest burst time is executed first. Execution Order: P4 → P1 → P3 → P2

Gantt Chart:



Process	CT	TAT	WT
P1	9	9	3
P2	24	24	16
P3	16	16	9
P4	3	3	0

Average Waiting Time = $(3 + 16 + 9 + 0) / 4 = 7$ ms

Average Turnaround Time = $(9 + 24 + 16 + 3) / 4 = 13$ ms

Example 2: Non-Preemptive SJF Scheduling with Arrival Time

Process	AT	BT
P1	3	5
P2	0	4
P3	4	2
P4	5	4

Step-wise Selection:

- At time 0, only P2 is available → execute P2
- At time 4, P1 and P3 are available → choose P3 (smaller BT)
- At time 6, P1 and P4 are available → choose P4
- Finally execute P1

Execution Order: P2 → P3 → P4 → P1

Gantt Chart:

P2		P3		P4		P1	
0ms	4ms	4ms	6ms	6ms	10ms	10ms	15ms

Process	CT	TAT (CT-AT)	WT (TAT-BT)
P1	15	12	7
P2	4	4	0
P3	6	2	0
P4	10	5	1

Average Waiting Time = $(7 + 0 + 0 + 1) / 4 = 2$ ms

Average Turnaround Time = $(12 + 4 + 2 + 5) / 4 = 5.75$ ms

Advantages: Minimum average waiting time; short processes execute first, increasing throughput.

Disadvantages: Requires prior knowledge of burst time (impractical for interactive systems); can cause **starvation** of long processes if short processes keep arriving.

3. Shortest Remaining Time First (SRTF) — Preemptive SJF

The preemptive version of SJF. The CPU is always allocated to the process with the shortest remaining burst time. When a new process arrives, its burst time is compared with the remaining time of the current process; if smaller, the current process is preempted.

SRTF achieves optimal average response time among all scheduling algorithms. However, it suffers from the same limitation as SJF — burst times must be known in advance — and can cause starvation of long processes even more severely, since a constant stream of short processes will continuously preempt them.

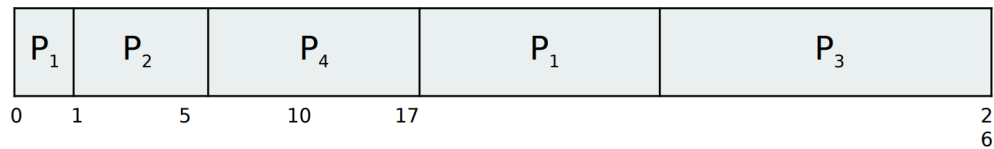
Example 1:

Process	AT	BT
P1	0	8
P2	1	4
P3	2	9
P4	3	5

SRTF is the preemptive version of SJF. At every instant, the process with the shortest remaining burst time is selected.

Execution Order:

- P1 executes from 0–1
- P2 arrives with smaller remaining time → preempts P1
- P2 executes from 1–5
- P4 executes from 5–10
- P1 executes from 10–17
- P3 executes from 17–26

Gantt Chart:

Process	CT	TAT (CT-AT)	WT (TAT-BT)
P1	17	17	9
P2	5	4	0
P3	26	24	15
P4	10	7	2

Average Waiting Time = $(9 + 0 + 15 + 2) / 4 = 6.5$ ms

Average Turnaround Time = $(17 + 4 + 24 + 7) / 4 = 13$ ms

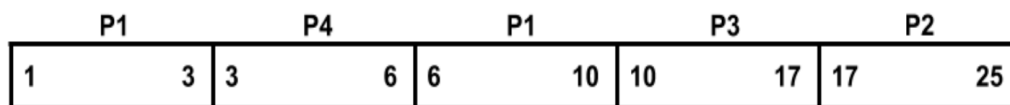
Example 2:

Process	AT	BT
P1	1	6
P2	1	8
P3	2	7
P4	3	3

Step-wise Execution:

- At time 1, P1 and P2 arrive → choose P1
- At time 2, P3 arrives, but P1 still has shortest remaining time
- At time 3, P4 arrives with BT = 3 → preempts P1
- After P4 completes, continue remaining shortest process

Execution Order: P1 → P4 → P1 → P3 → P2

Gantt Chart:

Process	CT	TAT (CT-AT)	WT (TAT-BT)
P1	10	9	3
P2	25	24	16
P3	17	15	8
P4	6	3	0

Total Waiting Time = $(3 + 16 + 8 + 0) = 27$ ms

Average Waiting Time = $27 / 4 = 6.75$ ms

Total Turnaround Time = $(9 + 24 + 15 + 3) = 51$ ms

Average Turnaround Time = $51 / 4 = 12.75$ ms

Advantages: Optimal average response time; short processes always served quickly.

Disadvantages: Starvation possible; requires future burst time knowledge; high context-switch overhead.

4. Priority Scheduling

Each process is assigned a priority number. The CPU is allocated to the process with the highest priority (conventionally, smaller integer = higher priority). It can be either preemptive or non-preemptive.

In **preemptive priority scheduling**, a newly arrived higher-priority process immediately preempts the currently running lower-priority one. In **non-preemptive priority scheduling**, the new process is placed at the head of the ready queue but waits for the current process to complete.

SJF is a special case of priority scheduling where priority is the inverse of the predicted next CPU burst time.

Starvation Problem: If high-priority processes keep arriving, low-priority processes may never execute. This is a major drawback. A famous example: when an IBM 7094 was shut down at MIT in 1973, a low-priority process submitted in 1967 was found — it had never run in 6 years.

Solution — Aging: The priority of a process is gradually increased the longer it waits. This ensures that every process eventually gets the CPU.

Example 1:

Process	Burst Time (BT)	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

Given: Smaller priority number indicates higher priority.

Execution Order: $P2 \rightarrow P5 \rightarrow P1 \rightarrow P3 \rightarrow P4$

Gantt Chart:



Process	CT	TAT	WT
P1	16	16	6
P2	1	1	0
P3	18	18	16
P4	19	19	18
P5	6	6	1

Average Waiting Time = $(6 + 0 + 16 + 18 + 1) / 5 = 8.2$ ms

Average Turnaround Time = $(16 + 1 + 18 + 19 + 6) / 5 = 12$ ms

Example 2: Priority Scheduling with Arrival Time

Process	AT	BT	Priority
P1	0	5	1
P2	1	3	2
P3	2	8	1
P4	3	6	3

Note: Higher priority number indicates higher priority.

Step-wise Selection:

- At time 0, only P1 is available → execute P1
- At time 5, P2, P3, and P4 are available
- P4 has highest priority → execute P4
- Next execute P2
- Finally execute P3

Execution Order: P1 → P4 → P2 → P3

Gantt Chart:

P1		P4		P2		P3	
0ms	5ms	5ms	11ms	11ms	14ms	14ms	22ms

Process	CT	TAT (CT-AT)	WT (TAT-BT)
P1	5	5	0
P2	14	13	10
P3	22	20	12
P4	11	8	2

Total Waiting Time = $(0 + 10 + 12 + 2) = 24$ ms

Average Waiting Time = $24 / 4 = 6$ ms

Total Turnaround Time = $(5 + 13 + 20 + 8) = 46$ ms

Average Turnaround Time = $46 / 4 = 11.5$ ms

Advantages: Critical/system processes (high priority) execute first.

Disadvantages: Starvation of low-priority processes; equal-priority processes require a tiebreaker algorithm.

5. Round Robin (RR)

Each process is assigned a fixed time slice called a **time quantum** (q), typically 10–100 ms. After the quantum expires, the process is preempted and added to the end of the ready queue. The next process in the queue then gets the CPU.

Round Robin is the algorithm most suited to time-sharing systems. It is inherently fair — each process gets $1/n$ of CPU time in chunks of at most q units, and no process waits more than $(n-1) \times q$ time units. It is completely free of starvation. Context switching is used to save the state of preempted processes.

Effect of Time Quantum Size:

Quantum Size	Effect
Very large ($\rightarrow \infty$)	Behaves like FCFS
Very small	Excellent response time but excessive context-switch overhead reduces effective CPU utilization
Optimal	Large enough to minimize context switch overhead while small enough to maintain good response time

The quantum should be large relative to context switch time but small enough to feel responsive. A rule of thumb: 80% of CPU bursts should be shorter than the quantum.

Example 1:

Process	Burst Time (BT)
P1	24
P2	3
P3	3

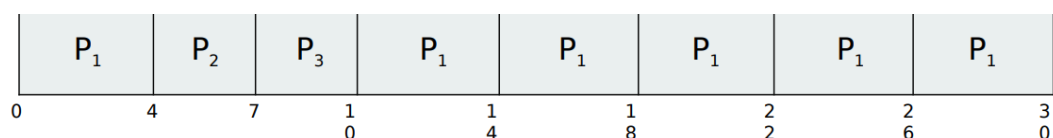
Time Quantum (q) = 4

In Round Robin scheduling, each process gets CPU for a fixed time quantum. If the process is not completed within the quantum, it is moved to the end of the ready queue.

Execution Order:

- P1 executes for 4 units $\rightarrow$ remaining BT = 20
- P2 executes and completes
- P3 executes and completes
- P1 continues until completion

Gantt Chart:



Process	CT	TAT	WT
P1	30	30	6
P2	7	7	4
P3	10	10	7

Average Waiting Time = $(6 + 4 + 7) / 3 = 5.67$ ms

Average Turnaround Time = $(30 + 7 + 10) / 3 = 15.67$ ms

Observation:

- Round Robin gives better response time compared to SJF.
- Time quantum should be large compared to context switching time.
- Typical time quantum ranges from 10 ms to 100 ms.

Example 2: Round Robin Scheduling with Arrival Time

Given: Time Quantum = 2 ms

Process	AT	BT
P1	0	10
P2	0	5
P3	0	8

Execution Sequence: P1 → P2 → P3 → P1 → P2 → P3 → P1 → P3 → P1 → P3 → P1

Gantt Chart:

P1		P2		P3		P1		P2		P3	
1	2	2	4	4	6	6	8	8	10	10	12

P1		P2		P3		P1		P3		P1	
12	14	14	15	15	17	17	19	19	21	21	23

Process	CT	TAT (CT-AT)	WT (TAT-BT)
P1	23	23	13
P2	15	15	10
P3	21	21	13

Total Waiting Time = $(13 + 10 + 13) = 36$ ms

Average Waiting Time = $36 / 3 = 12$ ms

Total Turnaround Time = $(23 + 15 + 21) = 59$ ms

Average Turnaround Time = $59 / 3 = 19.66$ ms

Time Quantum and Context Switching

Consider a process with execution time = 10 ms.

Case 1: Large Time Quantum

- Quantum = 12 ms
- Context Switch Time = 0
- Process completes in one CPU burst.

Case 2: Moderate Time Quantum

- Quantum = 6 ms
- Context Switch Time = 1
- Process requires two CPU cycles.

Case 3: Very Small Time Quantum

- Quantum = 1 ms
- Context Switch Time = 9
- Too many context switches occur, reducing CPU efficiency.

Conclusion:

- Very small time quantum increases context switching overhead.
- Very large time quantum makes RR behave like FCFS.
- An optimal quantum balances responsiveness and overhead.

Advantages: Fair allocation; no starvation; good response time; suitable for interactive and time-sharing systems.

Disadvantages: Higher context-switch overhead; larger average turnaround time than SJF; performance heavily depends on quantum choice.

Model Question

Ten independent instances of a program start execution (as ten different processes) at the same time. Each program repeatedly executes a loop 50 times. In each iteration, the program performs: CPU computation for 50 ms, followed by an I/O operation that takes 200 ms. The system uses Round Robin (RR) scheduling with a time quantum of 20 ms. The scheduling overhead is 3 ms per context switch. Assuming all ten processes arrive at time $t=0$, compute the response time of the first process. (6 Marks)

Solution

Given:

- Number of processes = 10
- Round Robin quantum = 20 ms
- Context switch overhead = 3 ms
- CPU burst = 50 ms
- All processes arrive at $t=0$

Response time: Time from arrival until the process gets CPU for the first time.

Step 1: Order of Execution

All 10 processes are in ready queue initially. The first process gets CPU immediately. So: Waiting before first execution = 0 ms

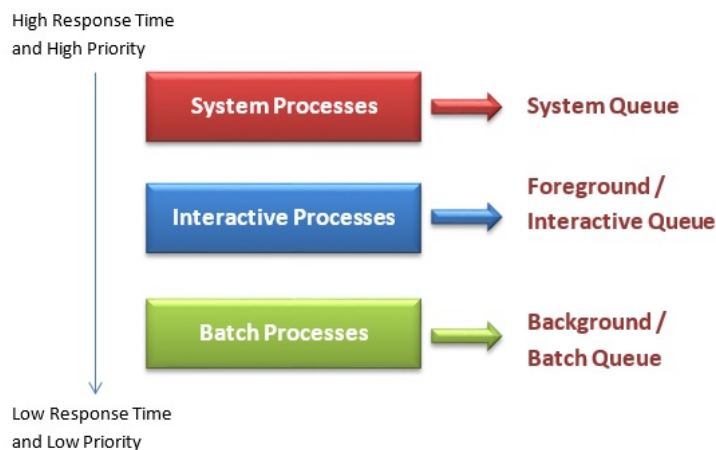
But context switch overhead occurs before execution.

Thus: Response Time = 3 ms

Multilevel Queue Scheduling (MLQ)

The ready queue is permanently divided into several separate queues, each for a specific class of processes, with each queue having its own scheduling algorithm.

The queues are arranged in strict priority order. System processes occupy the highest-priority queue, followed by interactive processes, then batch processes. Higher-priority queues are always served before lower-priority ones. A process is permanently assigned to one queue based on its type, memory size, or priority — it cannot move between queues. This inflexibility is the key limitation.



Each queue uses its own algorithm: for example, Queue 1 and 2 use Round Robin (for responsiveness), while Queue 3 uses FCFS (simpler, for batch jobs).

Disadvantages: Starvation of lower-priority queues if higher ones are always busy; inflexible because processes cannot migrate between queues.

Multilevel Feedback Queue Scheduling (MLFQ)

An extension of MLQ that allows processes to move between queues based on their observed CPU behavior. It is the most general and flexible CPU scheduling algorithm.

MLFQ addresses the key weakness of MLQ. Instead of permanently fixing a process to a queue, MLFQ promotes or demotes processes based on how much CPU they use. A process that consumes its entire time quantum is identified as CPU-bound and moved to a lower-priority queue. A process that frequently relinquishes the CPU (I/O-bound or interactive) is treated as high-priority and moved up. This way, the algorithm learns the nature of each process dynamically, without needing to know burst times in advance.

Basic Rules of MLFQ

- **Rule 1:** If $\text{Priority}(A) > \text{Priority}(B)$, A runs, B does not.
- **Rule 2:** If $\text{Priority}(A) = \text{Priority}(B)$, A and B run in Round Robin.
- **Rule 3:** When a new job enters the system, it is placed at the highest-priority queue.
- **Rule 4:** Once a job uses up its time allotment at a given level, its priority is reduced (moves down one queue).
- **Rule 5:** After some time period S, move all jobs to the topmost queue. (This prevents starvation of long-running processes — called **Priority Boost**.)

Advantages: No prior knowledge of burst time needed; automatically favors interactive/short processes; prevents starvation via aging/boost; adaptable to changing process behavior.

Disadvantages: Complex to configure (number of queues, quantum sizes, boost interval); vulnerable to gaming (a process could yield just before its quantum expires to stay in the high-priority queue).

Comparison of Scheduling Algorithms

Algorithm	Preemptive	Starvation	Optimal For	Key Weakness
FCFS	No	No	Short, uniform processes	Convoy effect
SJF	No	Yes	Minimizing avg. waiting time	Burst time must be known
SRTF	Yes	Yes	Minimizing avg. response time	Burst time must be known
Priority	Both	Yes	Systems with critical tasks	Starvation of low-priority
Round Robin	Yes	No	Time-sharing, interactive	High context-switch overhead
MLQ	Both	Yes (lower queues)	Mixed process types	Inflexible (no migration)
MLFQ	Yes	No (with boost)	General-purpose OS	Complex configuration

Case Study

Completely Fair Scheduler (CFS)

The Completely Fair Scheduler (CFS) was introduced in Linux kernel 2.6.23 to improve fairness and responsiveness in process scheduling. CFS is based on the idea that: Every runnable process should receive a fair share of CPU time. Instead of fixed time slices, CFS distributes CPU time proportionally among processes.

Core Idea of CFS

If there are n runnable processes: Each process should receive $1/n$ share of CPU time.

CFS always selects: The process that has received the least CPU time so far. This creates fairness among processes.

Process Priority in CFS

Linux uses **nice values** to represent priority.

Nice Value	Priority
-20	Highest priority
0	Default priority
+19	Lowest priority

Important Point:

- Lower nice value → more CPU time
- Higher nice value → less CPU time

Virtual Runtime (vruntime)

CFS tracks process execution using **vruntime**.

- Stored in sched_entity
- Represents CPU time consumed by process
- Process with smallest vruntime runs next

Core Rule of CFS: Run the process with the smallest vruntime.

Targeted Latency and Minimum Granularity

Targeted Latency

Time period during which every runnable process gets CPU access at least once.

Example: 4 processes → each gets 5 ms in 20 ms latency

Minimum Granularity

Prevents extremely small CPU allocations.

Default: 1 ms minimum CPU time

Red-Black Tree in CFS

CFS stores runnable processes in a **Red-Black Tree (RB Tree)** ordered by vruntime.

Important Points

- Leftmost node = smallest vruntime
- Leftmost process selected next

Operations

Operation	Function
Insert process	enqueue_entity()
Remove process	dequeue_entity()
Select next process	__pick_next_entity()

Note: Implementation details of RB tree are not required for syllabus.

Scheduler Classes in Linux

Linux uses different scheduler classes.

Scheduler	Purpose
SCHED_NORMAL	Normal processes using CFS
SCHED_FIFO	Real-time FIFO scheduling
SCHED_RR	Real-time Round Robin

Important Point: Real-time tasks have higher priority than normal tasks.

Sleeping and Waking Up

When a process waits for an event:

- It enters sleep state
- Removed from RB tree
- Another process is scheduled

Sleep States

State	Meaning
TASK_INTERRUPTIBLE	Can wake using signal
TASK_UNINTERRUPTIBLE	Ignores signals

Wake Up

When event occurs:

- wake_up() called - Process becomes TASK_RUNNING
- Reinserted into RB tree

Context Switching

Context switching means: Switching CPU from one process to another.

Handled by:

- context_switch()
- schedule()

Main Operations

Function	Purpose
switch_mm()	Switch memory address space
switch_to()	Save and restore CPU registers

need_resched Flag

Linux uses need_resched flag to indicate rescheduling requirement.

Set When

- Current process used fair CPU share
- Higher-priority process becomes runnable

Important Functions

Function	Purpose
set_tsk_need_resched()	Set flag
clear_tsk_need_resched()	Clear flag
need_resched()	Check flag

User Preemption

User preemption occurs when:

- Returning from system call
- Returning from interrupt handler

If need_resched is set:

- Scheduler selects another process

Kernel Preemption

Linux supports fully preemptive kernel.

A kernel task can be preempted if:

- It is safe
- No lock is held

preempt_count

- Tracks lock usage
- 0 → preemption allowed
- Nonzero → preemption unsafe

Kernel Preemption Occurs When

1. Interrupt handler exits
2. Lock released
3. schedule() called
4. Kernel task blocks

Lottery Scheduling

Lottery scheduling is a proportional-share scheduling algorithm.

Working

- Each process gets tickets
- Scheduler randomly selects winning ticket
- Process with winning ticket runs

Advantage

- Simple
- Fair over long execution

Disadvantage

- Not perfectly accurate in short runs

Stride Scheduling

Stride scheduling is deterministic proportional scheduling.

Working

- Each process assigned stride value
- Process with lowest pass value runs next

Advantage

- Exact CPU sharing

Disadvantage

- Complex handling of new processes

Property	Lottery	Stride
Correctness	Probabilistic	Exact
New Process Handling	Easy	Difficult
Global State	Minimal	Required
Accuracy	Approximate	Precise

Conclusion

The Linux Completely Fair Scheduler (CFS) provides fair CPU allocation by selecting the process with the smallest virtual runtime. Linux scheduling uses efficient mechanisms such as RB trees, preemption, context switching, and scheduler classes to improve responsiveness and performance. Features like kernel preemption and proportional-share scheduling make Linux highly efficient for modern multitasking systems.

MODEL QUESTIONS

1. On a Linux based system, a process holding an array of 1 million integers executes `fork()`. The child only reads the array to compute a sum, while the parent continues modifying the array. If the parent modifies 10% of the pages and the array spans 2000 pages, how many physical copies of the array exist immediately after `fork()`? How many pages get duplicated? (3 Marks)
2. On a system with n processors, what is the minimum and maximum number of processes that can be in the ready, running and blocked states?
3. Ten independent instances of a program start execution (as ten different processes) at the same time. Each program repeatedly executes a loop 50 times. In each iteration, the program performs: CPU computation for 50 ms, followed by an I/O operation that takes 200 ms. The system uses Round Robin (RR) scheduling with a time quantum of 20 ms. The scheduling overhead is 3 ms per context switch. Assuming all ten processes arrive at time $t=0$, compute the response time of the first process. (6 Marks)
4. A hypothetical operating system provides a system call that allows a process to request for memory allocation. An experienced programmer gives the following advice: "If your program makes many small requests for memory, you can improve its performance by combining them into a single request and making only one system call." Argue why the advice makes sense. (3 Marks)
5. If the following code runs successfully, how many new processes get created? Draw a process tree depicting the hierarchy among the created processes. (5 Marks)

```
int main(int argc, char *argv[])
{
    pid_t childpid = 0;
    int i;

    for (i = 1; i < 5; i++)
    {
        if ((childpid = fork()) ≤ 0)
            break;
    }

    return 0;
}
```

6. An operating system supports both user-level threads and kernel-level threads. A developer proposes the following guidelines for deciding which type of thread to use: G1: If a task is CPU-bound, it should be implemented as a kernel-level thread when the system has multiple processors; otherwise, it should be implemented as a user-level thread. G2: If a task is I/O-bound, it should be implemented as a user-level thread when the process has no kernel-level threads; otherwise, it should be implemented as a kernel-level thread. Do you agree or disagree with these guidelines? Briefly justify your answer (4 Marks)